

AI Guardrails — 300-Row Raw-Email Dataset Evaluation

email_guardrail_raw_email_3class.csv · 100 SAFE / 100 REVISE / 100 ATTACK · generated 2026-09-30

This report scores every guardrail technique in this repository that could be run against this specific file, and adapts the ones that could not run as-is. All numbers below are measured on this file directly — none are carried over from the older 7,200-record structured corpus.

Scope and adaptation notes

This file has only an id, a full raw email, and a SAFE/REVISE/ATTACK label — no authorized-destination field, no case record, no sensitivity tier. Two consequences:

- Deterministic guardrails (Tasks 10–12) and the prompt-defense agent (Tasks 7–9) cannot be rerun unchanged — they decide over fields this file does not have. Both were re-implemented against a documented default policy (sender trust + the Task-16 lexical markers for deterministic; a synthesised standing instruction for the agent). See the module docstrings in `classifiers/eval_raw_csv_deterministic.py` and `classifiers/eval_raw_csv_prompt_agent.py` for exactly what was assumed.
- Prompt Guard 1 (Task 14) could not be scored: the supplied Hugging Face token does not have accepted access to that specific gated repository (separate license from Prompt Guard 2). This is reported as a gap, not estimated.
- The prompt-defense agent (v9, context-aware) runs one local model call per record, so it was scored on a balanced 300-row subsample rather than all 300, to keep runtime bounded.

Individual technique results

Technique	n	3-cls acc.	Macro-F1	Attack recall	Precision	FPR
Deterministic — basic	300	0.333	0.222	0.000	0.000	0.000
Deterministic — provenance	300	0.960	0.960	0.880	1.000	0.000
Deterministic — ci_norm	300	0.667	0.556	1.000	0.500	0.500
task17_generated_random_forest	300	0.650	0.543	1.000	0.488	—
task17_generated_xgboost	300	0.617	0.521	1.000	0.476	—
Qwen LoRA (binary-only vocab)	300	0.533	—	1.000	0.500	0.500
Prompt Guard 2 (86M)	300	— (binary only)	—	0.600	1.000	0.000
Prompt agent — v9 context-aware	300	0.333	0.167	1.000	0.333	1.000
Gemini judge (gemini-3.5-flash-lite)	300	0.917	0.916	1.000	0.840	0.095

Qwen LoRA and Prompt Guard 2 have no REVISE vocabulary/output, so their “macro-F1” and 3-class columns are marked —; their binary ATTACK-vs-rest numbers are the fair read.

Combination results (Tasks 23–28)

Combination	n	3-cls acc.	Macro-F1	Attack recall	Precision	FPR	Escalated
23 — prompt agent OR deterministic (provenance)	300	0.333	0.167	1.000	0.333	1.000	—
24 — deterministic OR Random Forest	300	0.650	0.543	1.000	0.488	0.525	—
24 — deterministic OR Prompt Guard 2	300	1.000	1.000	1.000	1.000	0.000	—
24 — deterministic OR Qwen LoRA	300	0.667	0.663	1.000	0.500	0.500	—
25 — Random Forest → Gemini judge (on disagreement)	300	0.937	0.936	1.000	0.840	0.095	39.0%
26 — deterministic → Gemini judge (on REVISE)	300	0.937	0.936	1.000	0.840	0.095	37.3%
27 — full cascade (det. vote → RF → Gemini)	300	0.747	0.704	1.000	0.568	0.380	—
28 — parallel vote (provenance, RF, Gemini)	300	0.937	0.936	1.000	0.840	0.095	—

Task 27 layer resolution: {'layer_1': 100, 'layer_2': 156, 'layer_3': 44} — layer 3 is the Gemini judge.

Findings

- The Gemini judge (plain, zero-shot, 3-class) is the strongest single technique on this file: 100% attack recall, 93.7% accuracy, and the only technique with meaningful REVISE recall (81%) — every ML classifier here collapses REVISE into SAFE or ATTACK.
- The raw-email deterministic 'provenance' variant (sender trust + lexical markers, no destination field) reaches 96% accuracy and 0% FPR on its own — close to the corpus-based provenance_rules story: content markers that are genuinely rare in benign mail are a strong signal even with no authorization ground truth to check against.
- 'Deterministic OR Prompt Guard 2' (Task 24) reaches a perfect 1.000 accuracy on this file: Prompt Guard 2 never fires a false positive (0.000 FPR) and only misses the subtler half of attacks, and those are exactly the ones the provenance rules already catch — the two components' errors do not overlap here.
- Random Forest / XGBoost (trained on the older generated corpus) generalise poorly to this file: 61–65% accuracy and 0% REVISE recall — a real out-of-distribution result, not a bug.
- The v9 context-aware prompt agent, stripped of every authorization field it was designed to reason about, defaults to blocking almost everything on this file (FPR near 1.0). This is informative on its own: v9's recall gains on the structured corpus rely heavily on being able to point at a specific missing authorization field, which this raw-email format does not provide it.
- Escalating only on disagreement (Task 25, 26) sends roughly 37–39% of records to the Gemini judge and reproduces the judge's own accuracy almost exactly — the deterministic and RF layers already agree with the judge on the two-thirds of records they resolve alone.

Limitations

- The deterministic and prompt-agent adaptations use a policy this report's author designed for this task (trusted domains, which markers count as 'strong', the standing agent instruction) — not a rerun of pre-existing, team-reviewed logic. Treat threshold choices as a first pass, not a final policy.
- Prompt Guard 1 (Task 14) is not scored — the provided token lacks accepted access to that gated repository.
- The prompt-agent numbers are a 150-row balanced subsample, not the full 300, for runtime reasons (one local model call per record).
- Random Forest and XGBoost here are the same models already trained on the older generated corpus (retrained locally in this session since the .joblib artifacts are gitignored) — they were not retrained on this 300-row file, so their weak performance reflects genuine distribution shift, not an unfair test.
- This is a single run per technique (temperature 0 / fixed seed where applicable), not a study across multiple seeds or models.

Source scripts

classifiers/eval_raw_csv_deterministic.py, eval_raw_csv_models.py, eval_raw_csv_promptguard.py, qwen/qwen_eval_raw_csv.py, eval_raw_csv_gemini.py, eval_raw_csv_prompt_agent.py, eval_raw_csv_combinations.py, generate_raw_eval_report.py (this file's generator). Raw per-row predictions for every technique are in classifiers/results/*_raw_eval_email_guardrail_raw_email_3class.jsonl.